

האוניברסיטה העברית

בי"ס להנדסה ולמדעי המחשב

בחינה בקורסי מבוא למדעי המחשב:

67101 – מבוא למדעי המחשב - מואץ
67130 – מבוא למדעי המחשב - רגיל
67108 – מבוא למדעי המחשב לתלמידי תלפיות

תאריך: 11.3.2012
משך הבחינה: שלוש שעות

מועד ב' - סמסטר א' – תשע"ב – 2011-2012
פרופ' דני לישצ'ינסקי, פרופ' נעם ניסן

הבחיות:

- יש לענות על 3 מתוך 4 השאלות הבאות (3 בלבד). משקל כל השאלות זהה (34 נק'), אך הציון המקסימלי הוא 100.
- יש לכתוב פתרון לכל אחת מהשאלות שבחרת במחברת נפרדת. יש לציין על כריכת המחברת לאיזה שאלה המחברת מתייחסת, באופן קריא וברור.
- יש לציין את מס' תעודת הזהות ומספר המחברת על כל מחברת.
- הקפידו הקפדה יתרה על סדר וכתב יד קריא. בחינה בלתי קריאה לא תזכה את כותבה במלוא הנקודות.
- יש למחוק טיוטות באופן ברור.
- יש להשאיר שוליים.
- אסור להשתמש בעט אדום.
- הקפידו על סגנון תכנותי נאות: כתיבה באופן מודולרי, ללא שיכפול קוד וכו'.
- תעדו כל תכנית כך שניתן יהיה להבינה בנקל (מותר לתעד בעברית).
- פתרון מיטבי לשאלה יחשב פתרון פשוט ויעיל ככל האפשר. תוכנית מסורבלת או מסובכת לא תתקבל כתשובה מלאה (גם אם היא נכונה).
- אין להקצות או להשתמש במבני נתונים פרט לאלו שהוגדרו בכל שאלה (אלא אם כן צוין אחרת בשאלה). עם זאת, מותר להשתמש במבני נתונים נוספים אשר גודלם ומספרם אינו תלוי בגודל הקלט (למשל מותר להקצות מערך בן שלשה תאים פעם אחת).
- בכל השאלות ניתן להניח שהקלט חוקי ומקיים את תנאי השאלה. כמו כן, ניתן להניח כי המערכים והרשימות המקושרות הנתונות מכילים ערכים תקינים ואינם NULL, או מערכים בגודל 0, אלא אם מובהר בשאלה אחרת.
- בפתרון השאלה הינכם רשאים לבצע שימוש בשיטות שכבר מימשם בסעיפים אחרים של אותה השאלה (אך לא בשיטות משאלות אחרות!)

שאלה 1 (34 נקודות)

נתונה המחלקה הבאה לייצוג עץ בינארי:

```
public class Tree {
    private int data;
    private int size; // number of nodes in this subtree
    private Tree left, right;
    private Tree parent; // the parent of this subtree
    public Tree(int data) {
        this.data = data;
        size = 1;
        left = right = parent = null;
    }
    void balancedInsert(int value) { . . . }
    void mirror() { . . . }
    int countNodesAtLevel(int level) { . . . }
}
```

א. (13 נקודות) עץ בינארי מאוזן הוא עץ שבו לכל צומת הגדלים של שני תתי העצים שלו (מספר הצמתים שכל אחד מהם מכיל) נבדלים זה מזה ב-1 לכל היותר. עליכם לממש (ללא שימוש ברקורסיה) את השיטה `balancedInsert`. שיטה זו מוסיפה ערך חדש לעץ בינארי תוך שמירה על היותו מאוזן ועדכון כנדרש של כל שדות מבנה הנתונים. בפרט, אם מתחילים מעץ מאוזן ומוסיפים אליו מספר כלשהו של ערכים באמצעות שיטה זו, לאחר סדרת ההוספות הנ"ל עדיין על העץ להיות מאוזן. **שימו לב:** אין דרישה שהעץ יהיה עץ חיפוש, הדרישות הן אך ורק לגבי איזון.

ב. (3 נקודות) מהי הסיבוכיות של השיטה שמימשתם (הביעו באמצעות n , כאשר n הוא מספר הערכים בעץ).

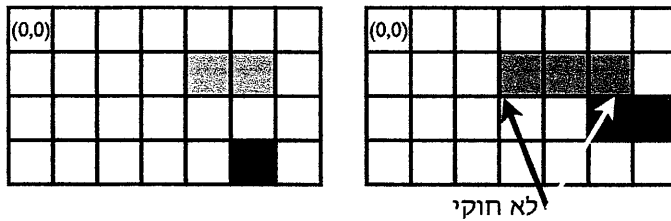
ג. (7 נקודות) ממשו את השיטה `mirror` אשר משנה את העץ לשיקוף ראי של עצמו (כלומר עבור כל צומת בנו הימני הופך להיות בנו השמאלי ובנו השמאלי הופך להיות בנו הימני). מותר להשתמש ברקורסיה. מומלץ להתחיל בכך שתציירו דוגמה לא טרייאלית (עץ המכיל 5 ערכים לפחות והשיקוף שלו).

ד. (7 נקודות) ממשו את השיטה `countNodesAtLevel` אשר מחזירה את מספר הצמתים בעץ בעומק בדיוק `level` מן הצומת שעליו נקראה השיטה. **כאן לא ניתן להניח כי העץ מאוזן.** מותר להשתמש ברקורסיה. לצורך העניין, נגדיר שכל צומת נמצא בעומק 0 מעצמו.

ה. (4 נקודות) חשבו את סיבוכיות כל אחת מהשיטות אשר כתבתם בסעיף ג' ו-ד'.

שאלה 2 (34 נקודות)

במשחק "צוללות" (battleships) ישנו שדה קרב המתואר על ידי לוח משבצות מלבני ואוסף ספינות קרב שאף הן מלבניות (בגדלים שונים) ואשר אסור שיהיה ביניהם מגע כלשהו: בין כל שתי ספינות מפרידה לפחות שורה או עמודה של משבצות. בציור שלהלן מוצג משמאל לוח משחק שעליו ממוקמות באופן חוקי מספר ספינות. מימין מוצג אותו הלוח עם דוגמאות למיקומי ספינות בלתי חוקיים.



נתון לכם הממשק הבא המתאר את הספינות במשחק:

```
interface Ship {  
    int getMinRow(); // הפינה השמאלית העליונה  
    int getMinCol(); // הפינה השמאלית העליונה  
    int getMaxRow(); // הפינה הימנית התחתונה  
    int getMaxCol(); // הפינה הימנית התחתונה  
    void recordHit(int row, int col); // פגיעה במשבצת הנתונה  
    boolean isDestroyed(); // כבר נפגעו  
}
```

א. (22 נקודות סה"כ) עליכם לממש את המחלקה BattleField המייצגת את שדה הקרב במשחק. מותר לכם להקצות מבני נתונים בעלי גודל כרצונכם, אך אין להשתמש ב-Collections. הניחו כי המתודות recordHit ו-isDestroyed מתבצעות ב- $O(1)$ כל אחת. עליכם לממש את המפורט להלן תחת הדרישה שהמתודה hit תהיה יעילה ככל האפשר, אך המתודה getRemainingShips לא חייבת להיות יעילה.

1. (4 נקודות) הגדירו את המחלקה וממשו בנאי המקבל כפרמטרים את הרוחב ואת הגובה של לוח המשבצות.

2. (8 נקודות) ממשו מתודה שמוסיפה ספינה ללוח המשחק. ההוספה תתבצע ותחזיר true רק אם המיקום חוקי, אחרת אין שינוי בלוח המשחק ומוחזר false:

```
public boolean addShip(Ship ship)
```

3. (4 נקודות) ממשו מתודה המטפלת בפגיעה במשבצת בקואורדינטות (row,col). אם יש ספינה שנפגעה יש לעדכן אותה (באמצעות recordHit), אך אין להסיר ספינות שהושמדו מן הלוח.

```
public void hit(int row, int col)
```

4. (6 נקודות) מתודה המחזירה מערך המכיל אך ורק את הספינות שטרם הושמדו (אין צורך לשכפל את הספינות):

```
public Ship[] getRemainingShips()
```

ב. (6 נקודות) נתחו את הסיבוכיות של כל אחת מן המתודות שממשתם.

ג. (6 נקודות) תארו אם וכיצד היה משתנה מימוש המחלקה והמתודות לו הייתם נדרשים למימוש יעיל עד כמה שאפשר עבור getRemainingShips, גם אם הדבר בא על חשבון יעילות הפעולות האחרות. בסעיף זה אין צורך לכתוב קוד, אלא להסביר בלבד (באופן רהוט).

שאלה 3 (34 נקודות)

נתונים הממשק הבא והמחלקה האבסטרקטית הבאה:

```
public interface Stream {      // מייצג רצף (סדרה) אינסופי של תווים
    public char next();        // מחזיר את התו הבא ברצף
}
```

```
public abstract class Encoder { // מייצג אלגוריתם לקידוד של תווים
    public abstract char encode(char c); // מחזיר את הקידוד של תו
}
```

א. (7 נקודות) כתבו מחלקה `InterleavedStream` המממשת את `Stream`, מקבלת בקונסטרקטור שלה שני אובייקטים מטיפוס `Stream` ומייצרת רצף חדש שמשלב את הרצפים הנתונים איבר, איבר. לדוגמא, אם נשלב את הרצף `a, b, c`, `a, ..., a` עם הרצף `x, y, x, y, ..., y` נקבל `a, x, b, y, c, x, y, a, x, y, ...`

ב. (6 נקודות) כתבו מחלקה `EncodedStream` המממשת את `Stream`, מקבלת בקונסטרקטור שלה אובייקט מטיפוס `Stream` וכן אובייקט מטיפוס `Encoder` ומייצרת רצף חדש המורכב מן התווים של הרצף הנתון כשהם מקודדים באמצעות המקודד הנתון.

ג. (4 נקודות) כעת הוסיפו למחלקה `Encoder` מתודה (לא אבסטרקטית)

```
public Stream encodeStream(Stream s)
```

המחזירה רצף המהווה קידוד של הרצף הנתון `s`.

ד. (7 נקודות) כתבו מחלקה `SubstitutionEncoder` היורשת מ-`Encoder` ומקבלת בקונסטרקטור שלה מפתח `key` שהינו מערך באורך 256 של `char` ומקודדת רצף על ידי החלפת כל תו `c` ב-`key[(int) c]`, כאשר ניתן להניח שכל התווים בזרם מיוצגים ע"י מספרים בין 0 ל-255. ניתן להניח שהמערך `key` לא ישתנה בעתיד ואין צורך להעתיקו.

ה. (10 נקודות) כתבו מחלקה `SubstitutionDecoder` היורשת מ-`Encoder` ומקבלת בקונסטרקטור שלה מפתח `key` שהינו מערך של `char` באורך 256 ומפענחת רצף שקודד על ידי `SubstitutionEncoder` עם מפתח זה. ניתן להניח כאן שהמפתח מתאר פונקציה שהיא חד-חד ערכית ועל (לכל ערך בין 0 ל-255 מתאים ערך אחד ויחיד בין 0 ל-255). כמו כן ניתן להניח שהמערך לא ישתנה בעתיד ואין צורך להעתיקו.

שאלה 4 (34 נקודות)

יהי A מערך המכיל מספרים שלמים (int). נאמר כי מערך A הינו "bitonic array" אם הערכים שבו מסודרים בסדר עולה ממש עד למקום מסוים במערך, וממקום זה ואילך הערכים מסודרים בסדר יורד ממש. כלומר: אם ב- A ישנם n מקומות, אזי קיים j כך שעבורו מתקיימים:

$$A[0] < A[1] < \dots < A[j]$$

וגם:

$$A[j] > A[j+1] > \dots > A[n-1]$$

שימו-לב כי מערך כנ"ל ייחשב ל-bitonic גם כאשר $j=0$ (כל המערך יורד ממש) או $j=n-1$ (כל המערך עולה ממש).

א. (12 נקודות) כיתבו שיטה סטטית:

```
public static boolean isBitonic(int[] a)
```

המקבלת מערך ומחזירה true אם המערך הינו bitonic array ו- false אחרת. נתחו את סיבוכיות השיטה.

ב. (11 נקודות) כיתבו שיטה סטטית:

```
public static int findMax(int[] a)
```

המקבלת מערך אשר ידוע עליו כי הוא bitonic array ומחזירה את ערכו של האיבר בעל הערך המקסימלי במערך. השיטה אמורה לעבוד בסיבוכיות $O(\log(n))$ כאשר n הוא גודל המערך.

ג. (11 נקודות) כיתבו שיטה סטטית:

```
public static void sortBitonic(int[] a)
```

המקבלת מערך אשר ידוע עליו כי הוא bitonic array וממיינת אותו בסיבוכיות זמן ריצה של $O(n)$, כאשר n הוא גודל המערך. בסעיף זה מותר להקצות מערך עזר באורך n . הסבר את אופן חישוב הסיבוכיות של השיטה שכתבת.